CASE STUDY

# GitLab, CVE-2026-97229: Lab-Verified Exposure and Fix

*A TeamStarWolf case study. Published 4 November 2026; every source checked as of that date.*

## Executive summary

Self-managed GitLab is where a great many teams keep their source code, their CI/CD pipelines and the secrets those pipelines use. That makes a GitLab server one of the most valuable hosts an organization runs, and it is often reachable from the internet so that developers and runners can talk to it; Shadowserver tracks nearly 2,800 internet-exposed GitLab CE instances, as BleepingComputer reports (**Reported** [13]). On 12 October 2026 GitLab shipped a critical patch release for a flaw in the way CI/CD configuration handles regular expressions: under certain conditions, an unauthenticated user could run arbitrary code on the server through a crafted regular expression (**Confirmed** [1]). GitLab scored it CVSS 3.1 10.0 [2] [3]. Seventeen days later CISA added it to the Known Exploited Vulnerabilities catalog [4].

The flaw is not really GitLab's own code. Ruby's security post of 23 October 2026 says CVE-2026-97229 and the earlier CVE-2026-93577 are the same bug: an integer overflow in the size of a compiled pattern in Ruby's regular expression engine, fixed upstream by Ruby pull request #19020 (**Confirmed** [5] [6]). GitLab closed it by patching the Ruby it bundles inside its Linux package (**Confirmed** [1]). That detail shapes everything a defender does next, and a lab run on 4 November 2026 showed why: a vulnerable 19.4.1 container and a fixed 19.4.2 container print the identical Ruby version string (**Demonstrated**, `02-test.txt` [33]), so on a packaged install the GitLab version, not `ruby -v`, is the signal to trust (**Inferred**).

This study sets out what the flaw is, the public record from the first incomplete fix in September to the KEV listing in October, what the lab showed, how to find and fix the flaw on real hosts, what GitLab says to look for in the logs, and the MITRE ATT&CK, CWE, CAPEC and D3FEND bindings with a Navigator layer.

## Key facts at a glance

| Field | Value |
|---|---|
| Identifier | CVE-2026-97229; GitHub advisory GHSA-9hj5-r43h-h3xg [3] [7] |
| Product | GitLab Community Edition and Enterprise Edition, self-managed; GitLab.com already patched, Dedicated tenants patched before publication [1] |
| Class | Integer Overflow or Wraparound (CWE-190) in the regular expression compiler used by CI/CD [2] [3] [4] [7] |
| Severity | CVSS 3.1 10.0 Critical, `AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:H` (GitLab as CNA); NVD shows only this CNA score [2] [3] |
| Affected | 19.2 before 19.2.8, 19.3 before 19.3.5, 19.4 before 19.4.2 [1] [3] |
| Fixed | 19.2.8, 19.3.5, 19.4.2 (12 October 2026) [1]; the later CVE-2026-110264 needs 19.4.3, 19.5.1 or later [10] |
| Root cause | Ruby/Onigmo compiled-pattern size overflow, Ruby PR #19020, merged 25 September 2026 [5] [6] |
| Ruby fix | Present from Ruby 4.0.8, 3.4.12 and 3.3.13; Ruby recommends 4.0.9, 3.4.12 or 3.3.13 [5] |
| Exploitation | CISA SSVC "active" on 13 October 2026 [3]; in CISA KEV since 29 October 2026, due 1 November 2026 [4] |
| Attribution | None: no opened primary names an actor [1] [2] [3] [4] [5] |
| Public exploit | Metasploit module, PR #22017, merged 30 October 2026 (**Demonstrated** [11]) |
| Credit | joaxcar (Johan Carlsson), through GitLab's HackerOne program [1] [5] |
| Lab result | 19.4.1 and 19.4.2 differ in GitLab version, not in `ruby -v` (**Demonstrated** [33]) |

## Key findings

- **One critical among twenty-four.** The 19.4.2 patch release lists 24 security fixes, 1 Critical, 5 High, 14 Medium and 4 Low, and the Critical comes first: "Integer Overflow issue in CI/CD Regular Expression Compiler impacts GitLab CE/EE" (**Confirmed** [1]). GitLab gives insufficient regular expression validation as the cause (**Confirmed** [1]).
- **It is a Ruby bug shipped inside a product.** Ruby says the two GitLab CVEs share one root cause, fixed by PR #19020, and that the CVE ids are GitLab's (**Confirmed** [5]). NVD's configuration lists Ruby 3.3.0 to before 3.3.13, 3.4.0 to before 3.4.12 and 4.0.0 to before 4.0.8 as vulnerable alongside GitLab (**Confirmed** [2]). CISA's KEV notes say other products using the same Ruby/Onigmo code may be affected (**Confirmed** [4]).
- **It was the second try.** GitLab 19.4.1, released on 23 September 2026, fixed the authenticated CVE-2026-93577 (CVSS 9.9) (**Confirmed** [8] [9]). GitLab's own detection metadata says 19.4.1 carries an incomplete fix that CVE-2026-97229 bypasses (**Confirmed** [8] [19]).
- **Exploited, unattributed.** CISA's SSVC entry rated exploitation active on the day the CVE was published, and KEV listed it on 29 October 2026 with forensic triage required under BOD 26-04 (**Confirmed** [3] [4]). No primary names a threat actor (**Confirmed** [4]). A public Metasploit module was merged on 30 October 2026 (**Demonstrated** [11]).

- **"Unauthenticated" needs a footnote.** GitLab, NVD and KEV say an unauthenticated user can trigger it. The Metasploit pull request says the attacker first needs an account that can push to a public project, after which an unauthenticated CI lint request triggers the overflow (**Reported** [11]).
- **The Ruby version string does not move.** In the lab both the vulnerable and the fixed image print `ruby 3.3.12 (2026-07-16 revision 0581089df9)`, with identical manifest checksums (**Demonstrated**, `02-test.txt` [33]). A scanner keyed on the NVD Ruby range would flag the fixed GitLab too (**Inferred** from that reading).
- **19.4.2 is not the end of the story.** The later CVE-2026-110264, another regular expression compiler integer overflow at CVSS 10.0, was fixed in 19.4.3 and 19.5.1 on 21 October 2026, so the lab's "fixed" 19.4.2 is still inside that range (**Confirmed** [10]).

### Top recommendations

1. Upgrade self-managed GitLab to 19.4.3, 19.5.1 or a later release now. That covers CVE-2026-97229 and CVE-2026-110264. For CVE-2026-97229 alone the floor is 19.2.8, 19.3.5 or 19.4.2, and 19.4.1 is not enough [1] [10] [19].
2. Decide exposure by the GitLab version (`/opt/gitlab/version-manifest.txt`, the `gitlab-ce` or `gitlab-ee` package version, Help, or `GET /api/v4/version`), never by `ruby -v` on a packaged install [15] [16] [33].
3. Hunt with GitLab's six published detections, starting with Workhorse 502s on CI lint and GraphQL requests that never reached the Rails logs [1] [19].
4. If any sign of code execution turns up, rotate the secrets in `gitlab-secrets.json`, all CI/CD variables, runner tokens and access tokens, because the code runs as the account that can read them [19].
5. For every other Ruby application, move to Ruby 3.3.13, 3.4.12 or 4.0.9, and stop compiling regular expressions from untrusted strings; Ruby says `Regexp.timeout` does not protect and length limits are not a reliable defense [5].

## Scope and method

This study covers CVE-2026-97229 in GitLab CE and EE, its sibling CVEs in the same Ruby code where they change what a defender must do, and the public record up to 4 November 2026. It does not cover the exploit's internals. The public Metasploit module is recorded by its metadata, its stated prerequisites and its stated effects only; it was not run or quoted, and its technique is not described [11]. Every web source was opened on 4 November 2026, and quotations are under 15 words. The GitLab work item and the two HackerOne reports that the CVE record cites could not be opened: NVD tags the work item Broken Link and the API returned 404, and the HackerOne reports redirect to sign-in, which NVD tags Permissions Required; the CNA tags those reports technical-description and exploit (**Confirmed** [2]).

The lab capture is the study's own evidence. It ran on a lab host on 4 November 2026, starting at 05:09 UTC with cleanup at 05:26 UTC, with two throwaway Docker containers bound to loopback, one per image, and only version strings were read [33].

| Grade | Meaning |
|---|---|
| Confirmed | Taken from a primary source that was opened: GitLab, NVD, the CVE record, CISA, the Ruby project, the GitHub Advisory Database or Docker Hub |
| Reported | A secondary or third-party account, not independently corroborated (press coverage, or a third party's statement about preconditions) |
| Demonstrated | A reproduction exists that others can check: the public exploit module, or this study's lab capture, with the capture file named |
| Inferred | The study's own reasoning from the cited facts, labelled as such |

# Part 1: The flaw

## What GitLab says

GitLab's patch release describes a flaw in which, under certain conditions, an unauthenticated user could execute arbitrary code on the GitLab server through a crafted regular expression in a CI/CD configuration, and it gives insufficient regular expression validation as the cause (**Confirmed** [1]). GitLab, acting as the CVE Numbering Authority, scores it CVSS 3.1 10.0 with a changed scope; NVD shows only that CNA (Secondary) score and has no NVD Primary score, and the record is Analyzed, last modified on 2 November 2026 (**Confirmed** [2] [3]). The weakness is CWE-190 on the CVE record (CNA title "Integer Overflow or Wraparound in GitLab"), on NVD, in the KEV entry and in GHSA-9hj5-r43h-h3xg; GitLab's own patch page names no CWE (**Confirmed** [2] [3] [4] [7]). The GitHub advisory, published on 13 October 2026, is critical at 10.0, lists no package ranges and is not GitHub-reviewed (**Confirmed** [7]).

GitLab's patch page lists no workaround and strongly recommends that all self-managed installations upgrade immediately (**Confirmed** [1]).

## What Ruby says

The bug lives in the regular expression engine that ships with Ruby, Onigmo (**Confirmed** [5]). Ruby's security post of 23 October 2026, by k0kubun, says CVE-2026-97229 and CVE-2026-93577 are the same bug, an integer overflow in the size of a compiled pattern, fixed by Ruby PR #19020, and that the fix is present from Ruby 4.0.8, 3.4.12 and 3.3.13 (**Confirmed** [5]). The CVE ids are GitLab's; Ruby did not request its own (**Confirmed** [5]).

The upstream pull request is titled "Avoid overflow in the regexp compiler's program size arithmetic." It was written by hsbt and merged on 25 September 2026 as commit `cf35145fc46bc278e0ab0003478fc4afe6564819` (**Confirmed** [6]). An `int` product in the size check for unrolled repeats overflowed, which let the compiler's size limit be bypassed (**Confirmed** [6]).

GitLab's detection write-up gives the shape of the CVE-2026-97229 bypass in defender terms: the pattern compiles small as US-ASCII, passes the 1 MiB size check, and is then recompiled for UTF-8 at match time, past `INT_MAX` (**Confirmed** [22]).

Ruby's statement of who is affected is narrow and useful: an application is affected only when it compiles a regular expression from an attacker-controlled string, through `Regexp.new`, `Regexp.union`, `Regexp.compile` or interpolation (**Confirmed** [5]). Ruby adds that limiting the pattern length is not a reliable defense and that

`Regexp.timeout` does not protect (**Confirmed** [5]). A CI/CD system that evaluates user-written rules is exactly the kind of program that compiles patterns from untrusted strings (**Inferred**).

## The family of regexp CVEs

Ruby's post names two more GitLab-assigned Ruby regular expression CVEs. CVE-2026-89078 is a parser double free, fixed by PR #13332 (merged 16 May 2025); CVE-2026-110264 is an overflow of internal repeat ids, fixed by PR #16801 (merged 29 April 2026) (**Confirmed** [5]). Ruby recommends 4.0.9, 3.4.12 or 3.3.13, and released 3.3.13 as an exception to the security-only maintenance phase of the 3.3 series (**Confirmed** [5]).

| CVE | What it is | GitLab fix | Ruby fix | Source |
|---|---|---|---|---|
| CVE-2026-93577 | Integer overflow in regular expression compiler; authenticated, CVSS 9.9 (`PR:L`) | 19.4.1, 19.3.3, 19.2.7 (23 September 2026) | PR #19020 | [5] [8] [9] |
| CVE-2026-97229 | Same overflow, unauthenticated path; CVSS 10.0 | 19.4.2, 19.3.5, 19.2.8 (12 October 2026) | PR #19020 | [1] [5] |
| CVE-2026-110264 | Overflow of internal repeat ids; CVSS 10.0 | 19.4.3, 19.5.1 and backports (21 October 2026) | PR #16801 | [5] [10] |
| CVE-2026-89078 | Parser double free | Not checked | PR #13332 | [5] |

NVD describes CVE-2026-93577, published on 24 September 2026, as an authenticated arbitrary-code-execution integer overflow (CWE-190) in CI/CD regular expression compilation, and it has no KEV listing (**Confirmed** [9]). It was also credited to joaxcar (**Confirmed** [8]).

The reading that CVE-2026-97229 is the successor to an incomplete 19.4.1 fix, closing an unauthenticated path where the earlier patch closed an authenticated one, began as the study's inference from Ruby's statement that both share one root bug (**Inferred** [5]). GitLab's detection metadata then states it directly: 19.4.1 contains an incomplete fix that CVE-2026-97229 bypasses (**Confirmed** [8] [19]).

## How GitLab shipped the fix

On the 19.4 line GitLab did not move its bundled Ruby to a new version: 19.4.1 and 19.4.2 both carry Ruby 3.3.12 (**Demonstrated**, `02-test.txt` [33]); the 19.3.5 and 19.2.8 images were not checked. Each of 19.4.2, 19.3.5 and 19.2.8 ships a bug-fix entry that adds a Ruby regexp program-size patch for Ruby 3.3 and 3.4, plus a separate repeat-range double-free patch, also under Bug fixes, which the page does not tie to any CVE (**Confirmed** [1]). The lab shows the consequence in Part 4: the patched GitLab still reports Ruby 3.3.12.

## The precondition conflict

GitLab, NVD and KEV all say "unauthenticated" (**Confirmed** [1] [2] [4]). The Metasploit pull request says something more specific: an attacker needs credentials for a user who can push to a public project, and an unauthenticated CI lint request then triggers the overflow; the resulting code runs as the `git` account, which GitLab's own detections also state (**Confirmed** [19]), and each attempt allocates about 2 GiB and crashes a Puma worker (**Reported** [11]). The two accounts can both be true if the unauthenticated step is the CI lint request and the low-privilege push account is a practical prerequisite of the public module (**Inferred**). For a defender the conservative reading is the vendor's: treat any internet-reachable instance as exposed, and treat open sign-up and broad public projects as factors that make exploitation easier (**Inferred**).

## Credit

GitLab credits the researcher joaxcar, who reported through GitLab's HackerOne bug bounty program, and Ruby names the discoverer of the PR #19020 overflow as Johan Carlsson (joaxcar) (**Confirmed** [1] [5]).

## Part 2: The public record

| Date (UTC) | Event | Grade | Source |
|---|---|---|---|
| 2025-05-16 | Ruby PR #13332 merged; Ruby later maps it to CVE-2026-89078, a parser double free | Confirmed | [5] |
| 2026-04-29 | Ruby PR #16801 merged; Ruby later maps it to CVE-2026-110264, an overflow of internal repeat ids | Confirmed | [5] |
| 2026-09-23 | GitLab 19.4.1, 19.3.3 and 19.2.7 fix the authenticated CVE-2026-93577 (CVSS 9.9); Docker Hub tag `gitlab/gitlab-ce:19.4.1-ce.0` pushed | Confirmed | [8] [14] |
| 2026-09-24 | CVE-2026-93577 published on NVD; CVE-2026-97229 reserved | Confirmed | [9] [3] |
| 2026-09-25 | Ruby PR #19020 merged upstream as `cf35145fc46b` | Confirmed | [6] |
| 2026-10-12 | GitLab 19.4.2, 19.3.5 and 19.2.8 critical patch release; Docker Hub tag `19.4.2-ce.0` pushed; BleepingComputer's first report gives 19.3.4 as the fixed 19.3 release | Confirmed; press Reported | [1] [14] [12] |
| 2026-10-13 | CVE published by GitLab at 01:05Z and by NVD at 02:17Z; GHSA-9hj5-r43h-h3xg published; CISA SSVC entry: Exploitation active, Automatable yes, Technical Impact total | Confirmed | [3] [2] [7] |
| 2026-10-14 | Metasploit PR #22017 opened by jheysel-r7, tested against `gitlab/gitlab-ce:19.4.1-ce.0` | Demonstrated | [11] |
| 2026-10-20 | GitLab's six detection rules for exploitation attempts dated (metadata last verified 2026-10-28) | Confirmed | [19] to [24] |
| 2026-10-21 | GitLab 19.5.1 and 19.4.3 fix CVE-2026-110264 (CVSS 10.0) | Confirmed | [10] |
| 2026-10-23 | Ruby security post ties both GitLab overflow CVEs to PR #19020 and recommends Ruby 4.0.9, 3.4.12 or 3.3.13 | Confirmed | [5] |
| 2026-10-29 | CISA adds CVE-2026-97229 to KEV; due 2026-11-01; known ransomware use Unknown; forensic triage required under BOD 26-04 | Confirmed | [4] |
| 2026-10-30 | Metasploit PR #22017 merged; BleepingComputer reports that CISA confirmed active exploitation and that GitLab reported attempts against self-managed instances | Demonstrated; press Reported | [11] [13] |
| 2026-10-31 | GHSA-9hj5-r43h-h3xg updated | Confirmed | [7] |
| 2026-11-01 | KEV due date for federal agencies | Confirmed | [4] |
| 2026-11-02 | NVD record last modified, status Analyzed, with the Ruby CPE ranges | Confirmed | [2] |
| 2026-11-03 | KEV catalog version 2026.11.03 does not list CVE-2026-93577 or CVE-2026-110264 | Confirmed | [4] |
| 2026-11-04 | Lab capture: vulnerable 19.4.1 and fixed 19.4.2 containers compared, then removed | Demonstrated | [33] |

### A version number that changed

BleepingComputer's first report on 12 October 2026 gave 19.3.4 as the fixed 19.3 release (**Reported** [12]). GitLab's page now names 19.3.5 and notes that 19.3.4 broke merge request pages when the Ona integration was enabled; the page carries no "updated on" note (**Confirmed** [1]). A change ticket or scanner rule written from that first report may name the wrong 19.3 target; treat 19.3.5 as the floor for the 19.3 line (**Inferred**).

## Part 3: Exploitation in the wild

CISA's KEV listing is the primary statement that the flaw is exploited. CISA added CVE-2026-97229 on 29 October 2026 as "GitLab Community Edition and Enterprise Edition Integer Overflow or Wraparound Vulnerability," with a due date of 1 November 2026, known ransomware use Unknown, forensic triage required under BOD 26-04, and a note that other products using the same Ruby/Onigmo code may be affected (**Confirmed** [4]). CISA's SSVC entry on the CVE record, dated 13 October 2026, already rated it Exploitation active, Automatable yes and Technical Impact total (**Confirmed** [3]).

GitLab's patch page publishes six threat detections for self-managed customers, but the page does not itself say that exploitation was observed (**Confirmed** [1]). BleepingComputer, in a 30 October 2026 article by Sergiu Gatlan, reports that GitLab reported exploitation attempts against self-managed instances and that Shadowserver tracks nearly 2,800 internet-exposed GitLab CE instances; the article names no threat actor (**Reported** [13]).

None of the opened primaries (GitLab, NVD, the CVE record, KEV and Ruby) attributes exploitation to a named threat actor (**Confirmed** [1] [2] [3] [4] [5]).

The public exploit is a Metasploit module, PR #22017 by jheysel-r7, opened on 14 October 2026, merged on 30 October 2026 and tested against `gitlab/gitlab-ce:19.4.1-ce.0` (**Demonstrated** [11]). It is recorded here only as destructive or weaponized: it runs attacker-chosen commands and crashes a worker on each attempt, and this study did not run it. With a public module and an automatable rating, the practical window between the patch and mass scanning is short (**Inferred** [3] [11]).

## Part 4: What the lab showed

The lab compared the last vulnerable 19.4 release with the first fixed one, using GitLab's own Docker images, and read only version information. No CI lint request, regular expression or payload was sent at any point, so the overflow was never triggered (**Demonstrated**, `plan.json`, `02-test.txt`, `04-mitigation.txt` [33]). The screenshots are renderings of the real command output with a neutral `lab$` prompt. Times are UTC.

### Setup

The run created an internal Docker network, `gl-lab-net`, and pulled `gitlab/gitlab-ce:19.4.1-ce.0` (vulnerable) and `gitlab/gitlab-ce:19.4.2-ce.0` (fixed), both 3.78 GB. HTTP was published only on 127.0.0.1, port 8929 for the vulnerable container and 8930 for the fixed one (**Demonstrated**, `01-setup.txt` [33]). The pulled digests were `sha256:9b33b45b9f42...b2809c` and `sha256:b65c3ece044d...4df11b`, and both match the digests Docker Hub's tag API lists for those tags, pushed on 23 September and 12 October 2026 (**Confirmed** [14]; **Demonstrated**, `01-setup.txt` [33]).

One deviation from the plan: the host had 7 GB of RAM and no swap, so the containers were started one after another. `gl-vuln` reached `(healthy)` at the fourth poll, about a minute after start, before `gl-fixed` was started (**Demonstrated**, `01-setup.txt` [33]).



## Test: the differing observable

The first line of `/opt/gitlab/version-manifest.txt` reads `gitlab-ce 19.4.1` in the vulnerable container and `gitlab-ce 19.4.2` in the fixed one, and `dpkg` inside the containers shows the packages `19.4.1-ce.0` and `19.4.2-ce.0` (**Demonstrated**, `02-test.txt`, `03-exposure.txt` [33]). A version-range check printed the verdict lines `gl-vuln 19.4.1 VULNERABLE to CVE-2026-97229` and `gl-fixed 19.4.2 not in affected range` (**Demonstrated**, `02-test.txt` [33]).

The bundled Ruby did not differ. Both images print `ruby 3.3.12 (2026-07-16 revision 0581089df9) [x86_64-linux]` from `ruby -v` and from `RUBY_DESCRIPTION`, and the manifest's `ruby` line carries the same version and the same sha256 in both (**Demonstrated**, `02-test.txt`, `04-mitigation.txt` [33]). `gitlab-rake gitlab:env:info` in the vulnerable container reported Ruby Version 3.3.12 and GitLab Version 19.4.1 (**Demonstrated**, `02-test.txt` [33]). This fits GitLab shipping the fix as a source patch on its bundled Ruby rather than a Ruby upgrade (**Inferred** from [1] and [33]).

One contradiction of the plan: the plan expected the loopback sign-in check to return 200 or 302. The first two attempts returned `HTTP/1.1 502 Bad Gateway` while Docker already reported the container healthy, and only a retry 75 seconds later returned `HTTP/1.1 200 OK` (**Demonstrated**, `02-test.txt` [33]). A Docker health status of healthy did not mean GitLab was ready to answer requests (**Inferred**). It is also a reminder that a Workhorse 502 is ordinary during start-up, which matters when reading the detection in Part 6 (**Inferred**).

## Exposure check

The exposure checks were run on the lab host itself, as they would be on any server, and then inside the two containers as examples of a packaged install. On the host they found no GitLab: the version manifest and the embedded Ruby were missing, `dpkg` printed nothing, and `rpm` answered `package gitlab-ce is not installed`; `docker ps` listed only the two lab containers (**Demonstrated**, `03-exposure.txt` [33]). The authenticated `/api/v4/version` check was skipped because the run had no token and the lab rules allowed requests only to loopback (**Demonstrated**, `03-exposure.txt` [33]).

The host's own system Ruby is `ruby 3.3.8 (2025-04-09 revision b200bad6cd)`, which is inside NVD's Ruby range of 3.3.0 to before 3.3.13 (**Demonstrated**, `03-exposure.txt` [33]; **Confirmed** [2]). Per Ruby's statement that matters only for applications on that host that compile regular expressions from untrusted strings [5].

CVE-2026-97229 exposure check: GitLab and Ruby versions

```
== CVE-2026-97229 exposure check (run on the lab host itself, and inside the two lab containers as Omnibus examples)
lab$  head -n 1 /opt/gitlab/version-manifest.txt
head: cannot open '/opt/gitlab/version-manifest.txt' for reading: No such file or directory

lab$  /opt/gitlab/embedded/bin/ruby -v
bash: line 1: /opt/gitlab/embedded/bin/ruby: No such file or directory

lab$  dpkg -l | grep -E 'gitlab-(ce|ee)'

lab$  rpm -q gitlab-ce gitlab-ee
package gitlab-ce is not installed
package gitlab-ee is not installed

lab$  docker ps --format '{{.Names}} {{.Image}}' | grep -i gitlab
gl-fixed gitlab/gitlab-ce:19.4.2-ce.0
gl-vuln gitlab/gitlab-ce:19.4.1-ce.0

# skipped: curl -s -H PRIVATE-TOKEN ... https://<your-gitlab>/api/v4/version -- placeholder host, no admin token, ar
lab$  ruby -v
ruby 3.3.8 (2025-04-09 revision b200bad6cd) [x86_64-linux-gnu]

lab$  docker exec gl-vuln head -n 1 /opt/gitlab/version-manifest.txt
gitlab-ce 19.4.1

lab$  docker exec gl-fixed head -n 1 /opt/gitlab/version-manifest.txt
gitlab-ce 19.4.2

lab$  docker exec gl-vuln dpkg -l | grep -E 'gitlab-(ce|ee)'
ii  gitlab-ce               19.4.1-ce.0                 amd64        GitLab Community Edition (including NGINX, Pc

lab$  docker exec gl-fixed dpkg -l | grep -E 'gitlab-(ce|ee)'
ii  gitlab-ce               19.4.2-ce.0                 amd64        GitLab Community Edition (including NGINX, Pc
```

## Mitigation check

The mitigation check compares each container's GitLab version with the fixed release using `sort -V`. It printed `gl-vuln 19.4.1 < 19.4.2 (upgrade required)` and `gl-fixed 19.4.2 >= 19.4.2 (patched release)`, and then printed the same Ruby description from both containers (**Demonstrated**, `04-mitigation.txt` [33]). The check verifies the version only. It does not show that the overflow is unreachable, because nothing was sent to trigger it (**Demonstrated**, `plan.json` [33]).




## Cleanup

The plan's `docker volume prune -f` was replaced by `docker rm -fv`, so that only the six anonymous volumes of the two lab containers were removed, rather than every unused volume on the host. Afterwards there were no containers, no `gl-lab-net`, no `gitlab/gitlab-ce` images and zero matching volumes, and the lab working folder was removed (**Demonstrated**, `05-cleanup.txt` [33]).

CVE-2026-97229 cleanup: containers, network, images removed

```
== CVE-2026-97229 cleanup  2026-11-04T05:26:36Z
lab$  docker inspect -f '{{.Name}} {{range .Mounts}}{{.Name}} {{end}}' gl-vuln gl-fixed
/gl-vuln 267217875f5bb304a6368debe10a46a707f69b3177fe8322114d1d64e4c20b80 835a5df6f6096426abb5fb0c61eab785ad2b2f7c49
/gl-fixed 1ae83ea2bfb02ece0d88f272181eddd34d8f5babe624f77f7f8737d48a4cdb56 374cd915008c6ebbce71ee17f0863392fd4c0f337

lab$  docker rm -fv gl-vuln gl-fixed
gl-vuln
gl-fixed

lab$  docker network rm gl-lab-net
gl-lab-net

lab$  docker rmi gitlab/gitlab-ce:19.4.1-ce.0 gitlab/gitlab-ce:19.4.2-ce.0
Untagged: gitlab/gitlab-ce:19.4.1-ce.0
Untagged: gitlab/gitlab-ce@sha256:9b33b45b9f42d176bada85ee5ecb81ddab7e506c435f44cd582206e284b2809c
Deleted: sha256:9bbf2223cbbae4c78854a65094e7ee00a9464796b48257d3d0b716a2154278c4
Deleted: sha256:3d21758f379afe2e32f5757027952de8792efdb4f43c5506ec5af95698a02d51
Deleted: sha256:1042bc1acf6cf8612852a25852f30489db07abd85995ec8c9ced16ae5cb6714c
Deleted: sha256:529e059cc180d7ad2d803e961e101a5f63c9669c7b6a703c7e9f8b140af24e2a
Deleted: sha256:89baa8ed6d3cc6c312aa3230ad832ad121ea507c1b8da4ae51de5cfe3c10c0df
Deleted: sha256:91f30e0930ac76cdf2b69b437f11926df0b7fc685cbc697083c65666407c4eb3
Deleted: sha256:57f25d7da86ebafaf9cced4369c7988de3f5e2cca9002b158f7b951bdcf8d806
Deleted: sha256:fc8fadc5b407fbb2ae456f7974bda07d2a1aad543602e20a78dcbcde34e12ffc
Deleted: sha256:8050f042a38db78755ae1123b3a690fe717d61acfe693d111f74de395a20171d
Deleted: sha256:65e5d715ea720349e6475f91651a0956279fc295a45fd73aa04f1cec046281a1
Untagged: gitlab/gitlab-ce:19.4.2-ce.0
Untagged: gitlab/gitlab-ce@sha256:b65c3ece044d125254b4c4a99b10676bbd2502f05f8c628cb60628f44f4df11b
Deleted: sha256:0be3fd698d4e2afb23fa7d583caae2fe369876de3fbbc320ec13e4f792e3d26c
Deleted: sha256:96d83296c84af6d2ccb7bbf5c3b88be19c8bff8bc2efdbb54328b76b7594dbd3
Deleted: sha256:da8e9376888a799f281e37e401e5a9232362864a031204ed5fe1b58d7042506f
Deleted: sha256:7f1494e509eebe16bd68d5f628c748ef8ba54c9ac11f2d00d4639a20bc0a9349
Deleted: sha256:1dfa41de2bf7c3a2910d703bb3dafffb488603d2c7df5b74a67e2648d25630c9
Deleted: sha256:9342fc9f3ccbc28be4c09770612c40d54897855ea2406b8c4e1a13abb2df49da
Deleted: sha256:1550f5f3cfc4003bc8e4a5261de71e505760fc60502188edfa2815da8fa22392
Deleted: sha256:8ec5341cd2ce4519f6367afb85972a7dc8c7300ed89112a3ef57239b4d4a08b1
Deleted: sha256:47f3bf9bc6382987e6d74f80aeee9d3eb2efd6dd5824d084068d9833858793b0
Deleted: sha256:d81fb2a7c6ee3be05fdefaa7fd223be219c1ab172083ec1b994c3ecda33ac0a8

# plan step 'docker volume prune -f' replaced by 'docker rm -fv' above so only this plan's anonymous volumes are rem
lab$  docker ps -a
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES

lab$  docker network ls
NETWORK ID     NAME      DRIVER    SCOPE
c6196556e799   bridge    bridge    local
c41a82569086   host      host      local
c1d6680892f7   none      null      local

lab$  docker images gitlab/gitlab-ce
REPOSITORY   TAG       IMAGE ID   CREATED   SIZE

lab$  docker volume ls -q | grep -cE "267217875f5b|835a5df6f609|9d5e0bf407c3|1ae83ea2bfb0|374cd915008c|1b97818765c0"
0

lab$  rm -rf /tmp/lab/CVE-2026-97229/

lab$  ls -la /tmp/lab/
total 0
drwxrwxr-x  2 kali kali  40 Nov  4 00:26 .
drwxrwxrwt 16 root root 840 Nov  4 00:26 ..

lab$  docker ps -a
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
```

### What the lab does and does not show

It shows that a vulnerable and a fixed GitLab are told apart by the GitLab version and not by `ruby -v`, which is the same in both, and that the cleanup left nothing behind (**Demonstrated** [33]). It does not show exploitability, and it produced no detection signal, because no CI lint request or regular expression was sent (**Demonstrated**, `plan.json` [33]). The detection content in Part 6 therefore comes from GitLab's rules only.

## Part 5: How to find and fix it on real hosts

### Step 1: find every GitLab

Look in three places: hosts with the Linux package, Docker hosts running a `gitlab/gitlab-ce` or `gitlab/gitlab-ee` image, and Kubernetes clusters running the Helm chart. The commands below are the ones the lab ran on its own host (**Demonstrated** [33]); on your own servers they only read state.

```
# Linux package (Omnibus): the GitLab version, first line of the manifest
head -n 1 /opt/gitlab/version-manifest.txt
# Debian and Ubuntu package version
dpkg -l | grep -E 'gitlab-(ce|ee)'
# RHEL-family package version
rpm -q gitlab-ce gitlab-ee
# Docker hosts: GitLab containers and their image tags
docker ps --format '{{.Names}} {{.Image}}' | grep -i gitlab
```

On a Linux package install, `/opt/gitlab/version-manifest.txt` holds the GitLab version and the versions of every bundled library, and package versions follow `MAJOR.MINOR.PATCH-EDITION.OMNIBUS_RELEASE`, for example `19.4.2-ce.0` (**Confirmed** [15]). On a self-managed instance the web UI shows the version under Help > Help, and `GET /api/v4/version` with a `PRIVATE-TOKEN` header returns `version`, `revision`, `enterprise` and `plan` (**Confirmed** [16]). Run the API call only against your own instance with your own token.

### Step 2: read the right version

The decision is the GitLab version:

| GitLab line | Vulnerable to CVE-2026-97229 | Fixed for CVE-2026-97229 | Also fixed for CVE-2026-110264 |
|---|---|---|---|
| 19.2 | before 19.2.8 | 19.2.8 | Backport release; check GitLab's 19.5.1 page for the 19.2 number |
| 19.3 | before 19.3.5 | 19.3.5 (not 19.3.4) | Backport release; check GitLab's 19.5.1 page for the 19.3 number |
| 19.4 | before 19.4.2 | 19.4.2 | 19.4.3 |
| 19.5 | Not listed in the CVE-2026-97229 ranges [3] | n/a | 19.5.1 |

Sources: [1] [3] [10]. The backport numbers for CVE-2026-110264 on the 19.2 and 19.3 lines were not checked.

Do not decide by `ruby -v` on a packaged install. The lab's vulnerable and fixed images print the same Ruby 3.3.12 string (**Demonstrated** [33]), so `ruby -v` cannot tell 19.4.1 from 19.4.2. A vulnerability scanner that matches the NVD Ruby range (before 3.3.13) against the bundled Ruby will flag a fixed GitLab as vulnerable

and will not notice when it is upgraded (**Inferred** from [2] and [33]). Key the finding on the GitLab package or manifest version, and suppress the bundled-Ruby match on GitLab hosts once the GitLab version is current (**Inferred**).

### Step 3: upgrade

GitLab's patch page lists no workaround (**Confirmed** [1]). Upgrades of a single-node instance involve downtime while database migrations run (**Confirmed** [1]). Back up first. The Linux package installer backs up the database automatically unless `/etc/gitlab/skip-auto-backup` exists, and a full backup of your own is still the safer starting point (**Confirmed** [17]; the last clause **Inferred**).

Linux package, from GitLab's upgrade page (**Confirmed** [17]):

```
# Debian and Ubuntu (use gitlab-ee=<version>-ee.0 for Enterprise Edition)
sudo apt update && sudo apt install gitlab-ce=<version>-ce.0
# RHEL 8 or 9 family (.el8 or .el9)
sudo dnf install gitlab-ce-<version>-ce.0.el9
# Amazon Linux 2023
sudo dnf install gitlab-ce-<version>-ce.0.amazon2023
```

The same page gives the `yum` and `zypper` forms for Amazon Linux 2 and SUSE (**Confirmed** [17]).

Docker, from GitLab's Docker upgrade page (**Confirmed** [18]): back up, then `docker pull gitlab/gitlab-ce:<version>-ce.0`, `docker stop gitlab`, `docker rm gitlab`, and `docker run` again with the same volume mounts for `/etc/gitlab`, `/var/log/gitlab` and `/var/opt/gitlab`. With Compose, `docker compose pull` and then `docker compose up -d`. Pin the image to an exact tag, as the lab did, so that the version you deploy is the version you checked (**Inferred**).

The Helm chart and source-install upgrade pages were not checked. A self-compiled GitLab, or one on a custom Ruby runtime, must also upgrade Ruby itself to 3.3.13, 3.4.12 or 4.0.9 (**Confirmed** [10]).

### Step 4: confirm

After the upgrade, re-run Step 1 and compare the version with the table above. The lab's comparison is a reusable shape for a fleet check: read the first line of the manifest and test it against the fixed release with `sort -V` (**Demonstrated**, `04-mitigation.txt` [33]). Choose the target release from the table, not from the lab's 19.4.2, because 19.4.2 is still inside the CVE-2026-110264 range [10].

### Other Ruby applications

The same regular expression engine, Onigmo, ships in every Ruby release in the affected ranges (**Confirmed** [2] [5]). Ruby says applications are affected only when they compile a regular expression from an attacker-controlled string, and its workaround for those that cannot upgrade is to stop compiling untrusted patterns; `Regexp.timeout` does not protect and length limits do not reliably protect (**Confirmed** [5]). Upgrade to Ruby 3.3.13, 3.4.12 or 4.0.9 or later [5]. For a system Ruby, `ruby -v` is the starting point, as on the lab host's 3.3.8 [33], but a distribution may backport the fix without changing the upstream version, so check the distribution's own advisory before closing a finding (**Inferred**). Search application code for `Regexp.new`, `Regexp.union`, `Regexp.compile` and interpolated patterns fed by request data (**Inferred** from [5]).

### Interim measures while you schedule the upgrade

These are not vendor workarounds and do not fix the flaw (**Inferred**): restrict open sign-up and the creation of personal access tokens, review which projects are public and who can push to them (the Metasploit prerequisite [11]), and limit network exposure of the CI lint API and the GraphQL endpoint to the people and runners that need them.

## Part 6: Detection and response

GitLab published six detections for self-managed customers on its patch page [1]. They live in GitLab's public detection repository and carry ATT&CK technique tags. The log paths are those of the Linux package.

| Detection | Log | What to look for | ATT&CK tags | Source |
|---|---|---|---|---|
| CI lint Workhorse worker crash (the primary detection) | `/var/log/gitlab/gitlab-workhorse/current` | A 502 with `badgateway: failed to receive response: EOF` on `POST /api/v4/projects/:id/ci/lint` or `/api/graphql`, with large `read_bytes` (about 2 MB), and a `correlation_id` that never appears in `api_json.log` | T1190, T1499 | [19] |
| CI regexp guard rejection | `/var/log/gitlab/gitlab-rails/application_json.log` | WARN from `Gitlab::Ci::Build::Rules::Rule::Clause::RegexpMatcher`, with the full pattern in `extra.regexp`; guards exist from 19.3.x; catches probes, not the bypassing request | T1190 | [20] |
| GraphQL ciLint hostile regexp | `/var/log/gitlab/gitlab-rails/graphql_json.log` | `ciLint` requests whose `variables` field carries a hostile regular expression construct; inline arguments are logged as REDACTED | T1190 | [21] |
| CI regexp oversized non-ASCII path | `api_json.log` or `production_json.log` | A request path with a segment near the 255-byte limit containing a percent-encoded non-ASCII byte, the CVE-2026-97229 trigger commit; pair with a `git ls-tree` scan | T1190 | [22] |
| CI lint scripted client error burst | `api_json.log` | Bursts of 5xx on `POST ci/lint` from python, curl or wget style user agents; request content is FILTERED | T1190, T1595.002 | [23] |
| Onigmo native worker crash (corroborating) | `/var/log/gitlab/puma/puma_stderr.log`; `/var/log/gitlab/sidekiq/current` | Ruby `[BUG]` crash reports with Onigmo frames (`onig_compile`, `compile_tree`), and Sidekiq cluster kills for the pipeline vector; a successful exploit may leave no crash | T1190, T1499.004 | [24] |

All six are **Confirmed** as published by GitLab; each rule is dated 20 October 2026 and its metadata was last verified on 28 October 2026 [19] to [24]. Three practical notes (**Inferred**): the Workhorse rule's strongest signal is the gap between Workhorse and Rails, since the lab saw ordinary 502s during start-up [33]; the guard-rejection rule records the pattern verbatim, so treat those log lines as sensitive; and because a successful exploit may not crash anything [24], absence of crash reports is not evidence of safety.

### Response

GitLab's primary detection gives the response (**Confirmed** [19]): confirm the instance is on 19.4.2, 19.3.5, 19.2.8 or later (19.4.1 is not sufficient), and on any sign of code execution rotate the secrets in `gitlab-secrets.json`, all CI/CD variables, runner tokens and access tokens. Preserve the Workhorse and Rails JSON logs before rotating, so the timeline survives (**Inferred**). Federal agencies also owe the BOD 26-04 forensic triage that KEV requires (**Confirmed** [4]).

The lab never sent a CI lint request or a regular expression, so it produced no detection signal; it cannot be used to test these rules (**Demonstrated**, `plan.json` [33]).

## Part 7: The vulnerability-management view

- **Every signal pointed the same way, early.** A 10.0 score, a changed scope, no authentication in the vendor wording, an SSVC rating of active and automatable on the day of publication, a public module within three weeks, and a KEV listing with a three-day due date [2] [3] [4] [11]. Unlike cases where the base score and the exploitation signals disagree, there was nothing to reconcile here: this was an emergency change from 13 October 2026 (**Inferred**). Compare the Linux TCP use-after-free study, where a 9.8 had no KEV listing.
- **Two weeks of SSVC before KEV.** The SSVC record said "active" on 13 October and KEV followed on 29 October [3] [4]. A program that waits for KEV loses those sixteen days; reading the SSVC fields on the CVE record would have caught it earlier (**Inferred**).
- **The first fix was not the last.** 19.4.1, 19.4.2 and 19.4.3 each closed a regular expression compiler CVE within a month [8] [1] [10]. A remediation SLA that closes the ticket at the first fixed version will reopen it twice (**Inferred**). Track the product's latest patch release, not the first one that clears a single CVE.
- **The scanner can be wrong in both directions.** The NVD Ruby range turns a fixed GitLab into a false positive, and a bundled-library check cannot see GitLab's source patch at all (**Inferred** from [2] [33]). Findings on products that bundle a runtime need the product version as the deciding field. See the Vulnerability Prioritization Reference.
- **The blast radius is the secrets.** A GitLab server holds the secrets in `gitlab-secrets.json`, CI/CD variables, runner tokens and access tokens [19]. Code execution as the `git` user (**Confirmed** [19]; also stated in the Metasploit PR [11]) is a supply-chain foothold, not only a server compromise (**Inferred**). See Supply Chain Security and DevSecOps.

### Overstated and understated

- **Overstated:** "unauthenticated" as the whole story. The vendor, NVD and KEV use the word (**Confirmed** [1] [2] [4]), but the public module's own prerequisite is an account that can push to a public project (**Reported** [11]); an instance with closed sign-up and no public projects is harder to reach than the wording suggests, though no less in need of the patch (**Inferred**). Also overstated: a scanner finding keyed on the NVD Ruby range, which flags a fixed GitLab because its bundled Ruby string never changes (**Demonstrated** [33]; the scanner behavior **Inferred** from [2]).
- **Understated:** that 19.4.2 is "fixed". It closes this CVE but sits inside the range of CVE-2026-110264, another 10.0 in the same compiler (**Confirmed** [10]). Also understated: the sixteen days in which the CVE record's SSVC entry said exploitation was active before KEV listed it (**Confirmed** [3] [4]), and the reach of

the root bug beyond GitLab, which CISA's KEV note flags for every product using the same Ruby/Onigmo code (**Confirmed** [4]).

## Part 8: MITRE mappings

No ATT&CK group, software or campaign applies, because no opened primary attributes the exploitation to anyone (**Inferred** [4]). The techniques below are the ones the record supports. Each id was checked against the live ATT&CK page in the Source column and against the library's ATT&CK v20.0 data, with the tactic as `data/attack/technique_profiles.jsonl` records it, and each has a generated page [25] to [29] [32] [34]. ATT&CK for ICS and Mobile do not apply.

### ATT&CK Enterprise techniques

| ATT&CK id | Name | Tactic (v20) | Dated event or report | Basis | Source |
|---|---|---|---|---|---|
| T1190 | Exploit Public-Facing Application | Initial Access | GitLab patch release (2026-10-12); KEV listing (2026-10-29); GitLab detections (2026-10-20) | Confirmed: remote code execution on an internet-facing server in GitLab's and NVD's description; tagged by all six vendor detections | [1] [2] [4] [19] to [24] [25] |
| T1499 | Endpoint Denial of Service | Impact | CI lint Workhorse worker crash detection (2026-10-20) | Confirmed: vendor detection tag | [19] [32] [34] |
| T1499.004 | Application or System Exploitation | Impact | Onigmo native worker crash detection (2026-10-20); each attempt crashes a Puma worker (Metasploit PR, 2026-10-14) | Confirmed: vendor detection tag; the crash per attempt is Reported | [24] [11] [26] |
| T1595.002 | Vulnerability Scanning | Reconnaissance | CI lint scripted client error burst detection (2026-10-20) | Confirmed: vendor detection tag | [23] [27] |
| T1078 | Valid Accounts | Initial Access (also Persistence, Privilege Escalation, Stealth) | Metasploit PR prerequisite of an account that can push to a public project (2026-10-14) | Reported: third-party prerequisite that conflicts with the vendor's "unauthenticated" | [11] [28] |
| T1059.004 | Unix Shell | Execution | Code runs as the `git` account (Metasploit PR, 2026-10-14) | Inferred: not in any vendor tag | [19] [11] [29] |

The parent techniques T1595 and T1059 are context only.

## Group, software and campaign objects

| Object | Id | Relation to this study | Source |
|---|---|---|---|
| Groups | none | No primary names an actor; BleepingComputer names none either | [1] [2] [3] [4] [5] [13] |
| Software | none | The Metasploit module is a public tool, not an ATT&CK software object in the record | [11] |
| Campaigns | none | No ATT&CK campaign covers this exploitation | [4] |

## CWE and CAPEC

| Id | Name | Where it appears | Basis | Source |
|---|---|---|---|---|
| CWE-190 | Integer Overflow or Wraparound | CVE record, NVD, KEV and GHSA-9hj5-r43h-h3xg | Confirmed | [2] [3] [4] [7] [30] |
| CAPEC-92 | Forced Integer Overflow | Related on the CWE-190 page; the library's CAPEC-92 page lists CWE-190 | Mapping Confirmed; fit to this flaw Inferred | [30] [32] |

The library has no per-id CWE pages; see the CWE Reference.

## D3FEND countermeasures

The library's D3FEND data maps T1190 to Inbound Traffic Filtering (D3-ITF) and Inbound Session Volume Analysis (D3-ISVA), among others (**Confirmed** [32]). The live D3FEND API confirms all eight ids below [31]. Fitting the other six to this flaw is the study's own mapping from GitLab's fix and detections (**Inferred**).

| D3FEND id | Countermeasure | Fit to this study | Basis | Source |
|---|---|---|---|---|
| D3-SU | Software Update | Upgrade to 19.4.3, 19.5.1 or later; Ruby 3.3.13, 3.4.12 or 4.0.9 | Inferred | [1] [5] [10] [31] |
| D3-SWI | Software Inventory | Find every GitLab and read its product version (Part 5) | Inferred | [15] [33] [31] |
| D3-AEM | Application Exception Monitoring | Workhorse 502s and Onigmo `[BUG]` crash reports | Inferred | [19] [24] [31] |
| D3-PSA | Process Spawn Analysis | Unexpected children of Puma or Sidekiq running as `git` | Inferred | [19] [11] [31] |
| D3-PLA | Process Lineage Analysis | Same hunt, by parent-child chain | Inferred | [11] [31] |
| D3-ITF | Inbound Traffic Filtering | Limit who can reach CI lint and GraphQL until patched | Library data maps it to T1190 | [32] [31] |
| D3-ISVA | Inbound Session Volume Analysis | Bursts of 5xx on CI lint from scripted clients | Library data maps it to T1190 | [23] [32] [31] |
| D3-ACH | Application Configuration Hardening | Restrict sign-up, personal access tokens and public projects | Inferred | [11] [31] |

### Navigator layer

navigator/library-layers/gitlab-cve-2026-97229.json carries the six technique entries above (ATT&CK v20, domain `enterprise-attack`), listed on the Navigator layers page. Score 100 marks a technique bound by GitLab's own description or detection tags; score 50 marks the Reported T1078 and the Inferred T1059.004. Each entry's comment names the source numbers.

MITRE ATT&CK, CWE, CAPEC and D3FEND content is used under the terms recorded in THIRD_PARTY_NOTICES.md.

## Cross-cutting themes

- **The vulnerable code was a dependency inside the product.** GitLab's Critical was Ruby's regular expression engine, carried inside the Linux package [1] [5]. The same pattern, a library flaw that every product embedding the library inherits, is the subject of the Log4Shell case study; this case adds the twist that the product patched the library in place without changing its version string (**Inferred** from [1] [33]).
- **Version strings lie when vendors patch in place.** Asset inventory and scanners usually trust the library version. On a GitLab package the product version is the truth and the bundled Ruby version is not (**Demonstrated** [33]; the generalization **Inferred**).
- **Incomplete fixes come in sequences.** One root bug produced an authenticated CVE, an unauthenticated bypass of its fix, and a third overflow in the same compiler within a month [5] [8] [1] [10]. After a parser or compiler fix, expect a follow-up and plan to patch again (**Inferred**).
- **"Unauthenticated" is a claim to read closely.** The vendor's wording and the public module's prerequisites differ [1] [11]. The difference does not change the patch decision, but it does say where to harden: sign-up, tokens and public projects (**Inferred**).
- **Lab work can stay on the safe side of the line.** A version-only comparison of two pinned images answered the defender's real question (how do I tell fixed from vulnerable) without sending anything that could trigger the bug (**Demonstrated** [33]).

## Sources

All web sources were opened on 4 November 2026. Source 33 is the lab capture of the same date.

| # | Source | URL | Opened | Grade |
|---|---|---|---|---|
| 1 | GitLab, critical patch release 19.4.2, 19.3.5 and 19.2.8 (2026-10-12): severity table, description, credit, Ruby patch entries, downtime note, detections list, 19.3.4 note | docs.gitlab.com/releases/patches/patch-release-gitlab-19-4-2-released/ | 2026-11-04 | Confirmed |
| 2 | NVD record for CVE-2026-97229 (API): CNA score, CWE-190, GitLab and Ruby CPE ranges, Analyzed, last modified 2026-11-02, reference tags | services.nvd.nist.gov/rest/json/cves/2.0?cveId=CVE-2026-97229 | 2026-11-04 | Confirmed |
| 3 | CVE record for CVE-2026-97229 (CVE Services API): CNA title, affected and fixed versions, solution, reserved and published times, CISA SSVC entry | cveawg.mitre.org/api/cve/CVE-2026-97229 | 2026-11-04 | Confirmed |

| # | Source | URL | Opened | Grade |
|---|---|---|---|---|
| 4 | CISA Known Exploited Vulnerabilities catalog, JSON feed, catalog version 2026.11.03 | cisa.gov/sites/default/files/feeds/known_exploited_vulnerabilities.json | 2026-11-04 | Confirmed |
| 5 | Ruby, security news on the regexp fixes for the GitLab CVEs, by k0kubun (2026-10-23) | ruby-lang.org/en/news/2026/10/23/regexp-fixes-for-gitlab-cves/ | 2026-11-04 | Confirmed |
| 6 | Ruby PR #19020, "Avoid overflow in the regexp compiler's program size arithmetic" (merged 2026-09-25) | github.com/ruby/ruby/pull/19020 | 2026-11-04 | Confirmed |
| 7 | GitHub Advisory Database, GHSA-9hj5-r43h-h3xg (published 2026-10-13, updated 2026-10-31) | github.com/advisories/GHSA-9hj5-r43h-h3xg | 2026-11-04 | Confirmed |
| 8 | GitLab, patch release 19.4.1, 19.3.3, 19.2.7 (2026-09-23), CVE-2026-93577 | docs.gitlab.com/releases/patches/patch-release-gitlab-19-4-1-released/ | 2026-11-04 | Confirmed |
| 9 | NVD record for CVE-2026-93577 (API) | services.nvd.nist.gov/rest/json/cves/2.0?cveId=CVE-2026-93577 | 2026-11-04 | Confirmed |
| 10 | GitLab, patch release 19.5.1 (2026-10-21), CVE-2026-110264 and the Ruby upgrade note for self-compiled installs | docs.gitlab.com/releases/patches/patch-release-gitlab-19-5-1-released/ | 2026-11-04 | Confirmed |
| 11 | Rapid7 Metasploit Framework PR #22017 (opened 2026-10-14, merged 2026-10-30): metadata and stated prerequisites only; not run | github.com/rapid7/metasploit-framework/pull/22017 | 2026-11-04 | Demonstrated (module exists); Reported (its prerequisites) |
| 12 | BleepingComputer, Sergiu Gatlan, first report on the patch release (2026-10-12) | bleepingcomputer.com/news/security/gitlab-warns-of-max-severity-rce-flaw-in-regex-compiler-engine/ | 2026-11-04 | Reported |
| 13 | BleepingComputer, Sergiu Gatlan, report on exploitation (2026-10-30) | bleepingcomputer.com/news/security/hackers-now-exploiting-max-severity-gitlab-flaw-in-attacks/ | 2026-11-04 | Reported |
| 14 | Docker Hub tag API, `gitlab/gitlab-ce` tags `19.4.1-ce.0` and `19.4.2-ce.0` | hub.docker.com/v2/repositories/gitlab/gitlab-ce/tags/19.4.2-ce.0 and hub.docker.com/v2/repositories/gitlab/gitlab-ce/tags/19.4.1-ce.0 | 2026-11-04 | Confirmed |
| 15 | GitLab documentation, package information (version manifest, package version format) | docs.gitlab.com/administration/package_information/ | 2026-11-04 | Confirmed |
| 16 | GitLab documentation, version API | docs.gitlab.com/api/version/ | 2026-11-04 | Confirmed |
| 17 | GitLab documentation, upgrade a Linux package instance | docs.gitlab.com/update/package/ | 2026-11-04 | Confirmed |
| 18 | GitLab documentation, upgrade a Docker instance | docs.gitlab.com/update/docker/ | 2026-11-04 | Confirmed |
| 19 | GitLab detection, CI lint Workhorse worker crash (rule dated 2026-10-20, last verified 2026-10-28) | gitlab.com/gitlab-security-oss/tldr/-/blob/main/Detections/platforms/GitLab/Vulnerability%20Detections/ci_lint_workhorse_worker_crash.md | 2026-11-04 | Confirmed |

| # | Source | URL | Opened | Grade |
|---|---|---|---|---|
| 20 | GitLab detection, CI regexp guard rejection (rule dated 2026-10-20, last verified 2026-10-28) | gitlab.com/gitlab-security-oss/tldr/-/blob/main/Detections/platforms/GitLab/Vulnerability%20Detections/ci_regexp_guard_rejection.md | 2026-11-04 | Confirmed |
| 21 | GitLab detection, GraphQL ciLint hostile regexp (rule dated 2026-10-20, last verified 2026-10-28) | gitlab.com/gitlab-security-oss/tldr/-/blob/main/Detections/platforms/GitLab/Vulnerability%20Detections/graphql_cilint_hostile_regexp.md | 2026-11-04 | Confirmed |
| 22 | GitLab detection, CI regexp oversized non-ASCII path (rule dated 2026-10-20, last verified 2026-10-28) | gitlab.com/gitlab-security-oss/tldr/-/blob/main/Detections/platforms/GitLab/Vulnerability%20Detections/ci_regexp_oversized_non_ascii_path.md | 2026-11-04 | Confirmed |
| 23 | GitLab detection, CI lint scripted client error burst (rule dated 2026-10-20, last verified 2026-10-28) | gitlab.com/gitlab-security-oss/tldr/-/blob/main/Detections/platforms/GitLab/Vulnerability%20Detections/ci_lint_scripted_client_error_burst.md | 2026-11-04 | Confirmed |
| 24 | GitLab detection, Onigmo native worker crash (rule dated 2026-10-20, last verified 2026-10-28) | gitlab.com/gitlab-security-oss/tldr/-/blob/main/Detections/platforms/GitLab/Vulnerability%20Detections/onigmo_native_worker_crash.md | 2026-11-04 | Confirmed |
| 25 | MITRE ATT&CK, T1190 Exploit Public-Facing Application | attack.mitre.org/techniques/T1190/ | 2026-11-04 | Confirmed |
| 26 | MITRE ATT&CK, T1499.004 Application or System Exploitation (sub-technique of T1499 Endpoint Denial of Service) | attack.mitre.org/techniques/T1499/004/ | 2026-11-04 | Confirmed |
| 27 | MITRE ATT&CK, T1595.002 Vulnerability Scanning (sub-technique of T1595 Active Scanning) | attack.mitre.org/techniques/T1595/002/ | 2026-11-04 | Confirmed |
| 28 | MITRE ATT&CK, T1078 Valid Accounts | attack.mitre.org/techniques/T1078/ | 2026-11-04 | Confirmed |
| 29 | MITRE ATT&CK, T1059.004 Unix Shell (sub-technique of T1059 Command and Scripting Interpreter) | attack.mitre.org/techniques/T1059/004/ | 2026-11-04 | Confirmed |
| 30 | MITRE CWE, CWE-190 Integer Overflow or Wraparound (related CAPEC-92) | cwe.mitre.org/data/definitions/190.html | 2026-11-04 | Confirmed |
| 31 | MITRE D3FEND API, technique records for D3-SU, D3-SWI, D3-AEM, D3-PSA, D3-PLA, D3-ITF, D3-ISVA and D3-ACH | d3fend.mitre.org/api/technique/d3f:SoftwareUpdate.json | 2026-11-04 | Confirmed |
| 32 | The library's ATT&CK v20.0 data: `data/attack/technique_profiles.jsonl`, `data/attack/technique_to_d3fend.jsonl`, and the generated CAPEC-92 page | /mitre/capec/CAPEC-92.md | 2026-11-04 | Confirmed |
| 33 | Lab capture, CVE-2026-97229: `plan.json`, `01-setup.txt`, `02-test.txt`, `03-exposure.txt`, `04-mitigation.txt`, `05-cleanup.txt` and the screenshots, 2026-11-04, setup from 05:09 UTC and cleanup at 05:26 UTC | Lab capture (not published); screenshots in Part 4 | 2026-11-04 | Demonstrated |

| # | Source | URL | Opened | Grade |
|---|---|---|---|---|
| 34 | MITRE ATT&CK, T1499 Endpoint Denial of Service | attack.mitre.org/techniques/T1499/ | 2026-11-04 | Confirmed |

*© 2026 The MITRE Corporation. This work is reproduced and distributed with the permission of The MITRE Corporation. License: ATT&CK Terms of Use. ATT&CK® and MITRE ATT&CK® are registered trademarks of The MITRE Corporation. This project is not affiliated with or endorsed by MITRE.*

*© The MITRE Corporation. Includes material from the Common Weakness Enumeration (CWE™), used under the CWE Terms of Use. https://cwe.mitre.org/*

*© The MITRE Corporation. Includes material from the Common Attack Pattern Enumeration and Classification (CAPEC™), used under the CAPEC Terms of Use. https://capec.mitre.org/*

*© The MITRE Corporation. Includes material from MITRE D3FEND™, used under the D3FEND Terms of Use. https://d3fend.mitre.org/*

*The terms for every MITRE dataset used here are recorded in THIRD_PARTY_NOTICES.md.*

## Related references

- CVE Reference, CWE Reference, CAPEC Reference, D3FEND Reference
- Threat Actors, Malware Families, Notable Incidents: no actor or malware family is attributed to this exploitation
- Vulnerability Prioritization Reference, Vulnerability Management Reference, Triage a CVE
- DevSecOps, Supply Chain Security, Ruby
- Other lab-backed studies: Ghostcat, the Tomcat AJP file read in the lab; ProFTPD mod_copy; Magento and Adobe Commerce PolyShell; Node.js Brotli decoder and PHP 8.4.27, two more version checks that tell a vulnerable build from a fixed one; Nextcloud richdocumentscode, another product that bundles a component and fixed it in place
- Other vulnerability studies: Log4Shell, a library flaw inherited by every product that embeds it; the Linux TCP use-after-free, where a 9.8 had no KEV listing
- Techniques: T1190 Exploit Public-Facing Application, T1499 Endpoint Denial of Service, T1499.004 Application or System Exploitation, T1595.002 Vulnerability Scanning, T1078 Valid Accounts, T1059.004 Unix Shell
- CAPEC: CAPEC-92 Forced Integer Overflow
- D3FEND: Software Update, Software Inventory, Application Exception Monitoring, Process Spawn Analysis, Inbound Traffic Filtering, Inbound Session Volume Analysis, all countermeasures
- Navigator layers, including this study's layer, gitlab-cve-2026-97229.json
- All case studies